

1.connect连接第五个参数如果是BlockingQueuedConnection，且信号和槽都在一个线程内，会产生死锁吗？

```
1  else if (c->connectionType == Qt::BlockingQueuedConnection)
2  {
3      /*****1*****/
4      if (receiverInSameThread) {
5          qWarning("Qt: Dead lock detected while
6  activating a BlockingQueuedConnection: "
7  "Sender is %s(%p), receiver is %s(%p)",
8  sender->metaObject()->className(), sender,
9  receiver->metaObject()->className(),
10 receiver);
11     }
12     /*****2*****/
13     QSemaphore semaphore;
14     {
15         QBasicMutexLocker
16         locker(signalSlotLock(sender));
17         if (!c->receiver.loadAcquire())
18             continue;
19         /*****3*****/
20         QMetaCallEvent *ev = c->isSlotObject ?
21             new QMetaCallEvent(c->slotObj,
22             sender, signal_index, argv, &semaphore) :
23             new QMetaCallEvent(c->method_offset,
24             c->method_relative, c->callFunction,
25             sender,
26             signal_index, argv, &semaphore);
27         /*****4*****/
28         QApplication::postEvent(receiver, ev);
29     }
30     /*****5*****/
31     semaphore.acquire();
32     continue;
33 #endif
34 }
```

会导致死锁，此时流程参考上面代码，我定义了几个标记点，当同一个线程既是信号发出者又是接收者时

- 真实信号发出时，该线程走入该代码，由于是同一个线程，所以标志位 receiverInSameThread 为1，检测到死锁，打印警告信息
- 走到标记4处，分发事件，事件通过事件循环机制发送，并用**信号量保护起来**，然后走到标记5
- 此时 semaphore.acquire () 函数尝试获取信号量（该函数传参 int n，表示获取 n 个信号量，不足 n 时线程阻塞，直到有 n 后，消费 n 并退出阻塞状态），结果发现信号量不足被阻塞

如果信号和槽在两个线程

- 信号线程被阻塞，但是槽线程走到了标记3，判断自己是槽函数，走到 QMetaCallEvent 函数，处理完后让信号量增加，信号线程会解除阻塞

```

4   MainWindow::MainWindow(QWidget *parent)
5       : QMainWindow(parent)
6       , ui(new Ui::MainWindow)
7   {
8       ui->setupUi(this);
9       connect(this,SIGNAL(SIG_test()),this,SLOT(SLOT_test()),Qt::BlockingQueuedConnection);
10  }
11
12  MainWindow::~MainWindow()
13  {
14      delete ui;
15  }
16
17
18  void MainWindow::on_pushButton_clicked()
19  {
20      qDebug()<<__func__;
21      emit SIG_test();
22  }
23
24  void MainWindow::SLOT_test()
25  {
26      qDebug()<<__func__;
27      while(1){
28          //qDebug()<<"1";
29      }
30  }

```

应用程序输出

```

bqc_test
17:33:59: Starting F:\qt_5.12.11\IMServer_project\build-bqc_test-Desktop_Qt_5_12_11_MinGW_32_bit-Debug\debug\bqc_test.exe ...
on_pushButton_clicked
Qt: Dead lock detected while activating a BlockingQueuedConnection: Sender is MainWindow(0x73fe1c), receiver is MainWindow(0x73fe1c)

```

实测发现和理论一致，定义一个 push button，点击之后，在主线程中绑定的信号和槽没有调用槽函数，同时由于主线程被阻塞，再次点击 ui 里的 push button 会一直阻塞（无法处理事件分发中的事件了，点击也没反应）

2. InnoDB 和 MyISAM 的比较

1. MyISAM 只支持表锁，innodb 支持事务（然后分别从原子一致隔离持久讲其机制）和行锁
2. innodb 支持外键，MyISAM 不支持

3.InnoDB支持全文索引（类似文章找关键词）

4.MyISAM只支持读未提交和读已提交，InnoDB默认为可重复读

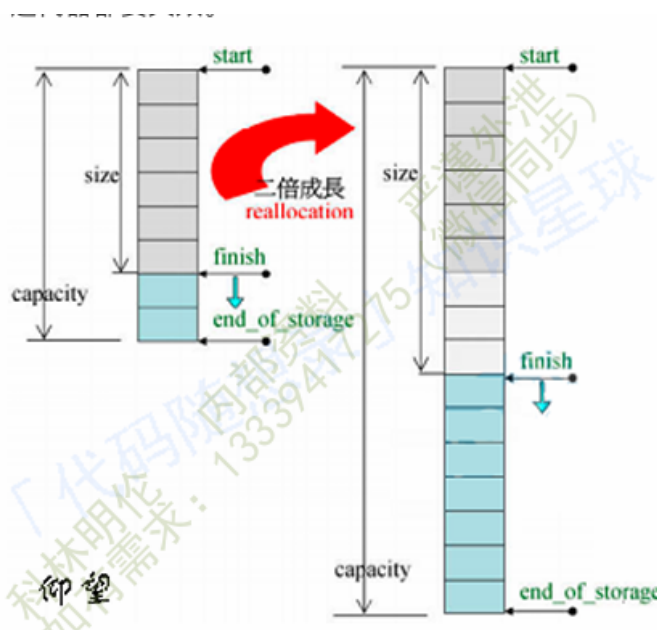
5.InnoDB支持聚簇索引和非聚簇索引（二级索引），MyISAM只支持非聚簇索引

3.介绍STL六大组件

容器：存储和管理数据的单元，本质是个类模版

序列式容器：vector, list, deque

- vector底层：一段连续的内存空间



1.size和capacity分别是什么，resize和reserve的作用

元素个数和容量，resize改变当前容器中的元素个数，reserve是改变当前容器的最大容量

如果resize (len) > capacity，则size和capacity都设置为len（**重新创建空间并拷贝**），否则size设置为len，capacity不变

如果reserve (len) > capacity，**则重新分配空间，把之前的拷贝过来销毁之前的内存**，否则不做任何操作

2.vector如何扩容

通过三个指针，start, finish, end_of_storage，如果所需新内存+finish超出end_of_storage，就通过allocate分配（stl空间配置器）所需内存**两倍大**的一块性内存，最后拷贝之前空间的数据（插入新值，假设有），并调整三个指针，销毁原来的空间

3.push_back和emplace_back的区别

push_back会先构造一个对象，然后再在内存中拷贝构造一个对象并放入容器

emplace_back直接在内存中拷贝构造一个对象并放入容器，具体实现是通过move转变为右值引用（变成参数），再搭配forward完美转发直接转交给空间适配器，避免隐式拷贝带来的额外开销

- list底层：双向循环链表，每一个节点都是一个随机内存空间

1.vector和list的比较

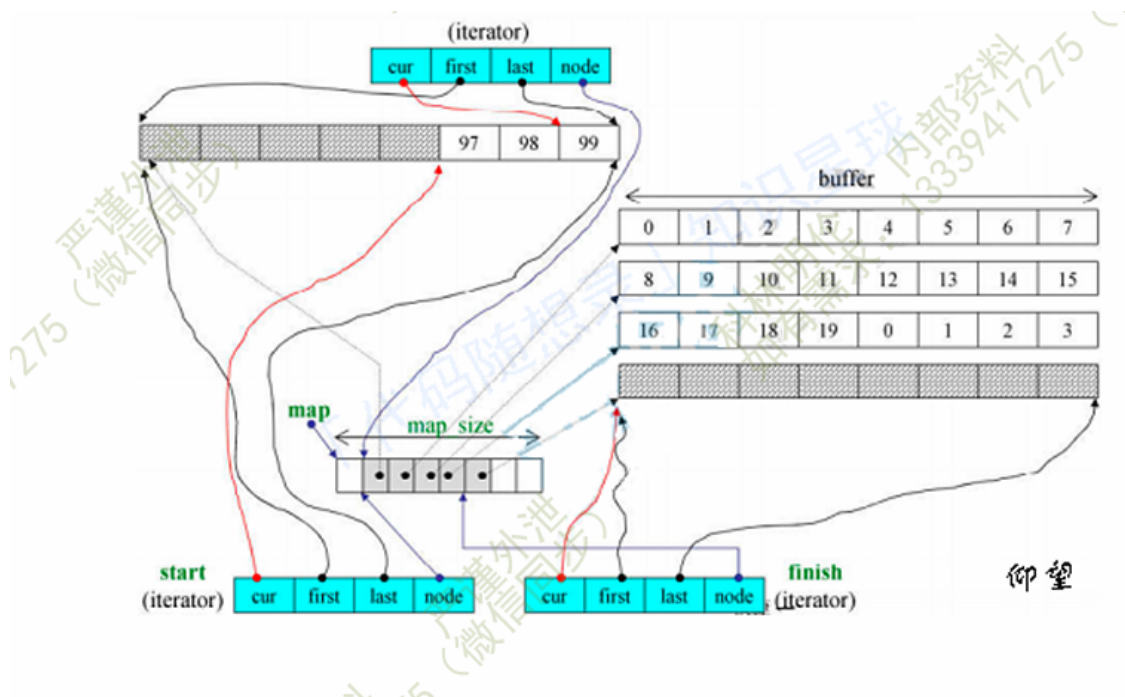
底层：连续空间和双向循环链表

节点访问：vector可以按下标寻址，list只能通过迭代器遍历

节点插入删除：vector插入删除涉及拷贝，list不涉及

扩容：vector会预分配内存，内存不够时翻倍扩容，list插入新节点时申请新内存

- deque底层：一个中控器map，内部是一个指针数组（但是支持前后扩容），指针数组中每个指针指向一片空间



1.数组插入尤其是头插头删效率很差，有什么办法提升数组插入删除的效率？

模仿deque的结构：

因为其一，deque头插时支持前向扩容（如果前面没有空间了），而不是像数组那样全部先拷贝到后面，空出第一个节点的位置，再插入

其二，deque假如在中间插入，也要移动后面的节点位置，但相比于数组移动真实节点（假如每个节点都是struct，一个节点100字节），移动指针肯定简单得多（4个字节）

关联性容器：map, set(红黑树), unordered_map (哈希表)

迭代器：可以看作是专门为了访问容器而设置的特殊指针（专业解释：提供统一访问接口，让算法和容器解耦）

分为输入迭代器，输出迭代器（这两个分别从输入流按指定格式（比如int）读取数据，和往输出流输出数据，咱用不到）

单向迭代器（也叫前向迭代器）

双向迭代器

随机访问迭代器

什么叫迭代器失效：

指对容器做某些操作，比如插入删除重新分配内存后，原有的迭代器不再指向有效的位置或元素，导致使用这些迭代器时出现未定义行为，导致程序崩溃，比如对vector插入元素，则该位置后面的所有元素的迭代器都失效，删除也是一样，扩容全部迭代器都失效

算法：对容器中数据操作的一系列函数模版

仿函数：可调用函数对象，内部operator()

比如sort函数中最后一个参数，表示从大到小或从小到达排序，就是仿函数

适配器：对现有的容器或迭代器进行封装，让他提供新的功能

容器适配器stack, queue（以deque为底层），priority_queue(以堆（以完全二叉树为底层）为底层）

迭代器适配器：比如reverse_iterator（反向迭代器）

分配器：为容器创建新节点，或扩容分配空间的

4.c++的STL库是线程安全的吗，什么时候不线程安全？项目中哪些地方用了STL库

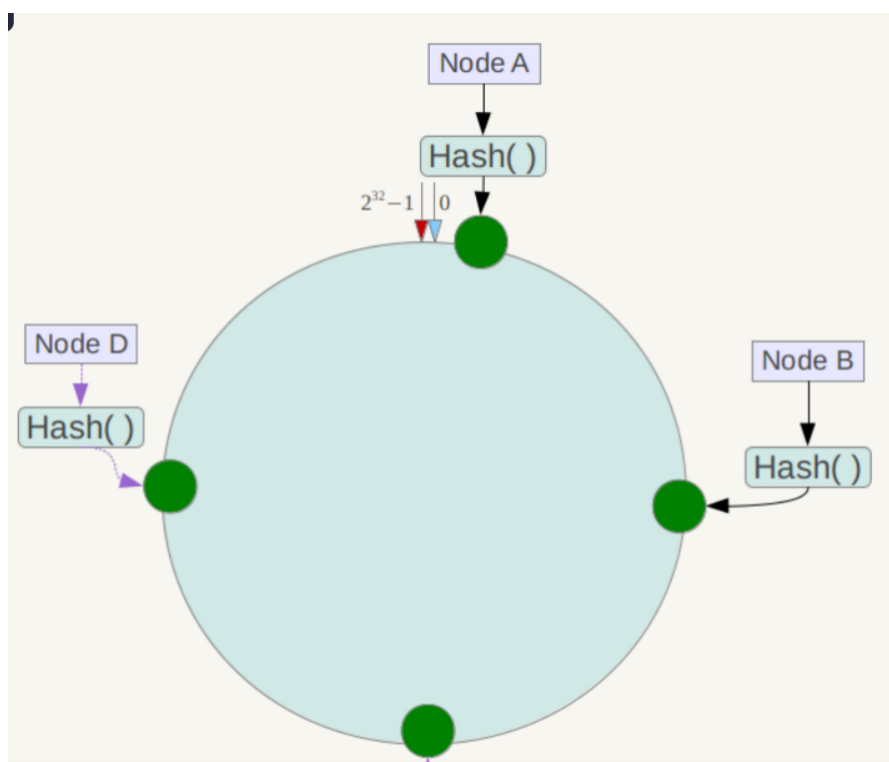
并非线程安全的，比如多线程同时修改一个容器，就会出现问题，所以一般在操作时我们都会重新封装STL库，在所有需要原子操作的地方加锁

比如在我的网盘项目中，我就在服务器那边封装了一个MyMap，并存储了用户id和用户信息的一个映射，以及文件时间戳到文件信息的一个映射，在音视频项目的客户端，封装了一个队列m_queue来存储音视频解码后的包，避免主线程放数据包和子线程取数据包的冲突

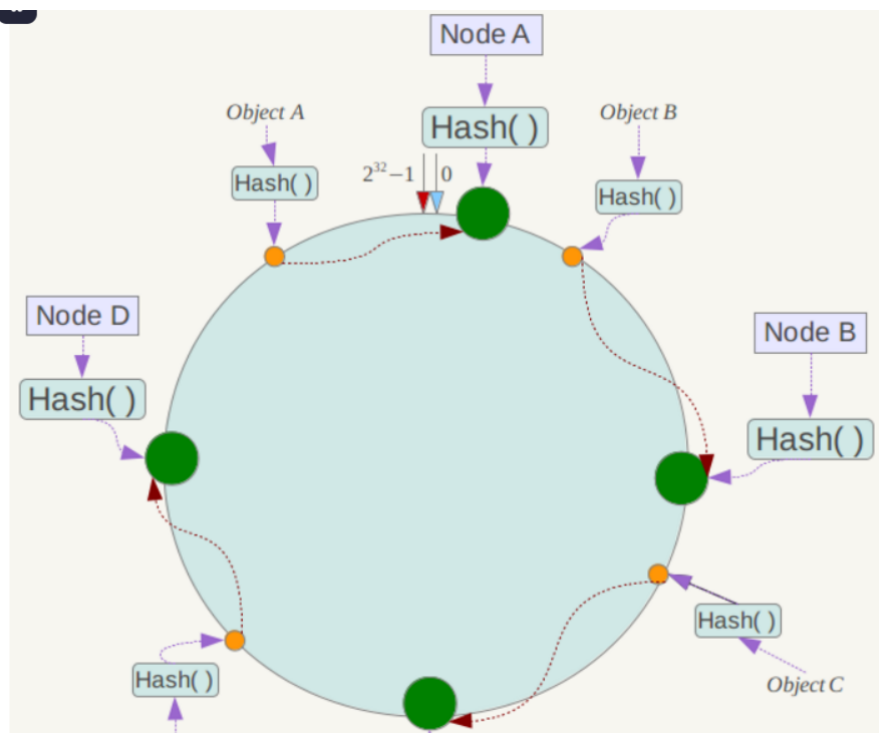
一致性哈希

一致性哈希考虑的是怎么高效的把数据分布到服务器集群中（**数据合理分配到服务器上，让每个服务器都负载一定量的数据**），且保证当前集群承载不下庞大数据量时，增加新的集群节点时可以更快融入该集群的问题

首先是哈希映射，一致性哈希根据服务器相关特征，比如ip地址，和特定规则，将服务器节点映射到一个环上（环大小一般为 $2^{32}-1$ ）

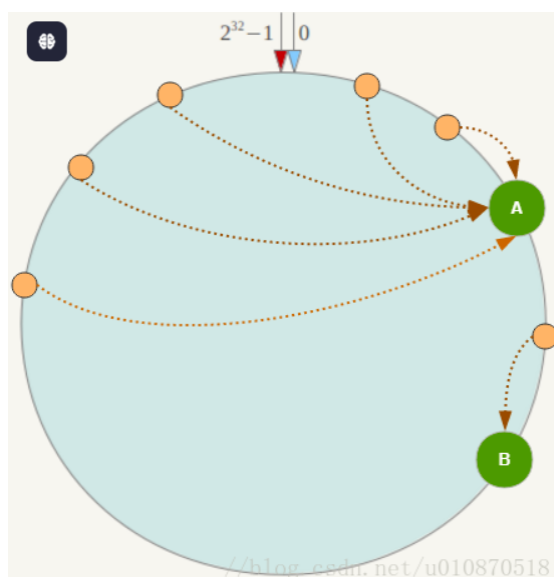


然后分配数据的时候，数据也通过哈希算法映射到环上，然后服务器节点与节点之间包含的数据就归该服务器所有

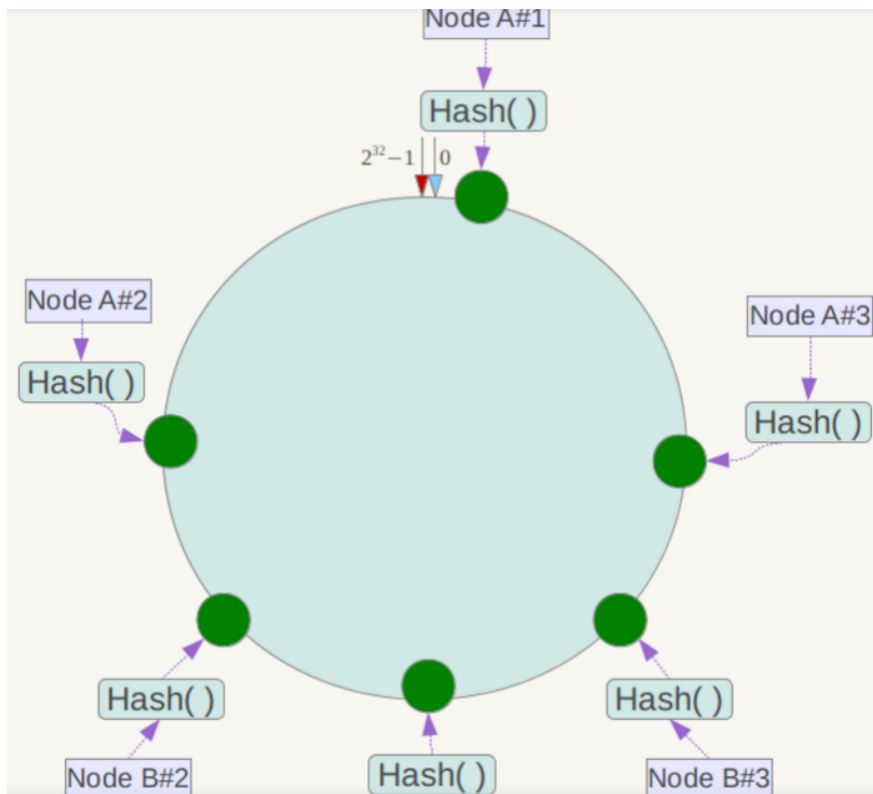


比如数据b归节点a所有，这种方式如果增加新的服务器节点，只需要找到其映射位置，然后把该位置到下一个节点间的数据复制到该节点同时删除这些数据在下一个节点中的备份即可。

最后就是分布不均的可能性，假如有如下情况



服务器a基本分走了所有数据，而b几乎没数据，对于这种情况，一致性哈希的手段是映射更多节点，比如a服务器，我可以给他映射到更多的位置（用别的哈希规则）



比如这里a有3个，那么a存储的数据就有三个小圆弧，这样就避免了数据倾斜的可能性